Status	Finished
Started	Friday, 30 October 2026, 9:43 PM
Completed	Friday, 30 October 2026, 9:57 PM
Duration	13 mins 48 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

A sentence is a list of words that are separated by a single space with no leading or trailing spaces. Each word consists of lowercase and uppercase English letters.

A sentence can be shuffled by appending the 1-indexed word position to each word then rearranging the words in the sentence.

For example, the sentence "This is a sentence" can be shuffled as "sentence4 a3 is2 This1" or "is2 sentence4 This1 a3".

Given a shuffled sentence s containing no more than 9 words, reconstruct and return the original sentence.

Example 1:

Input:

is2 sentence4 This1 a3

Output:

This is a sentence

Explanation: Sort the words in s to their original positions "This1 is2 a3 sentence4", then remove the numbers.

Example 2:

Input:

Myself2 Me1 I4 and3

Output:

Me Myself and I

Explanation: Sort the words in s to their original positions "Me1 Myself2 and3 I4", then remove the numbers.

Constraints:

2 <= s.length <= 200

s consists of lowercase and uppercase English letters, spaces, and digits from 1 to 9.

The number of words in s is between 1 and 9.

The words in s are separated by a single space.

s contains no leading or trailing spaces.

Answer: (penalty regime: 0 %)

```python
import sys

def main():
    # Read the input sentence from standard input
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    words = input_data.split()
    # Create a list to hold the words in their correct positions
    sorted_words = ["" for _ in range(len(words))]

    for word in words:
        # The last character is the 1-based index
        idx = int(word[-1]) - 1
        # The rest of the string is the actual word
        actual_word = word[:-1]
        sorted_words[idx] = actual_word

    # Join the sorted words with spaces and print
    print(" ".join(sorted_words))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	is2 sentence4 This1 a3	This is a sentence	This is a sentence	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

A sentence is a string of single-space separated words where each word consists only of lowercase letters.A word is uncommon if it appears exactly once in one of the sentences, and does not appear in the other sentence.

Given two sentences s1 and s2, return a list of all the uncommon words. You may return the answer in any order.

Example 1:

Input: s1 = "this apple is sweet", s2 = "this apple is sour"

Output: ["sweet","sour"]

Example 2:

Input: s1 = "apple apple", s2 = "banana"

Output: ["banana"]

Constraints:

1 <= s1.length, s2.length <= 200

s1 and s2 consist of lowercase English letters and spaces.

s1 and s2 do not have leading or trailing spaces.

All the words in s1 and s2 are separated by a single space.

Note:

Use dictionary to solve the problem

For example:

Input	Result
this apple is sweet this apple is sour	sweet sour

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read two sentences from standard input
    lines = sys.stdin.read().splitlines()
    if len(lines) < 2:
        return

    s1 = lines[0].strip()
    s2 = lines[1].strip()

    # Combine all words from both sentences into a single list
    all_words = s1.split() + s2.split()

    # Use a dictionary to count the frequency of each word
    word_count = {}
    for word in all_words:
        word_count[word] = word_count.get(word, 0) + 1

    # Find all words that appear exactly once across both sentences
    uncommon_words = [word for word, count in word_count.items() if count == 1]

    # Print the uncommon words separated by a space
    print(" ".join(uncommon_words))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	this apple is sweet this apple is sour	sweet sour	sweet sour	✓
✓	apple apple banana	banana	banana	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program that takes an input string from the user and counts the number of occurrences of each vowel (a, e, i, o, u) in the string. The program should be case-insensitive, meaning it should treat uppercase and lowercase vowels as the same.

Description:

Vowels play a significant role in the English language and other alphabet-based languages. Counting vowels in a given string is a fundamental task that can be applied in various text processing applications, including speech recognition, linguistic research, and text analysis. The objective of this problem is to create a Python script that accurately counts and displays the number of times each vowel appears in a user-provided string.

Program Requirements:

Input:

First line reading String as input, The string can contain any characters, including letters, numbers, and special characters.

Output:

Display the number of occurrences of each vowel in the string.

The output should list each vowel followed by its count.

Example:

Consider the following example for better understanding:

- **Input:** "Python Programming"
- **Output**

  a = 1
  e = 0
  i = 1
  o = 2
  u = 0

For example:

Input	Result
Hello World	a = 0 e = 1 i = 0 o = 2 u = 0
Python	a = 0 e = 0 i = 0 o = 1 u = 0

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input
    text = sys.stdin.read().strip()

    # Convert the string to lowercase for case-insensitivity
    text = text.lower()

    # Initialize a dictionary to store vowel counts
    vowel_counts = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

    # Count each vowel in the string
    for char in text:
        if char in vowel_counts:
            vowel_counts[char] += 1

    # Print the result in the specified format
    for vowel, count in vowel_counts.items():
        print(f"{vowel} = {count}")

```

```
if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	Hello World	a = 0 e = 1 i = 0 o = 2 u = 0	a = 0 e = 1 i = 0 o = 2 u = 0	✓
✓	AEIOU aeio u	a = 2 e = 2 i = 2 o = 2 u = 2	a = 2 e = 2 i = 2 o = 2 u = 2	✓
✓	Python	a = 0 e = 0 i = 0 o = 1 u = 0	a = 0 e = 0 i = 0 o = 1 u = 0	✓
✓	abcdefghijklmnopqrstuvwxyz	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓
✓	12345!@#$%AEIOU	a = 1 e = 1 i = 1 o = 1 u = 1	a = 1 e = 1 i = 1 o = 1 u = 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Create a student dictionary for n students with the student name as key and their test mark assignment mark and lab mark as values. Do the following computations and display the result.

1.Identify the student with the highest average score

2.Identify the student who as the highest Assignment marks

3.Identify the student with the Lowest lab marks

4.Identify the student with the lowest average score

Note:

If more than one student has the same score display all the student names

Sample input:

4

James 67 89 56

Lalith 89 45 45

Ram 89 89 89

Sita 70 70 70

Sample Output:

Ram

James Ram

Lalith

Lalith

For example:

Input	Result
4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Clean and split all lines, ignoring empty lines
    lines = [line.strip() for line in input_data.splitlines() if line.strip()]
    if not lines:
        return

    n = int(lines[0])
    students = {}

    for i in range(1, n + 1):
        if i >= len(lines):
            break
        parts = lines[i].split()
        if len(parts) < 4:
```

```
            continue
        name = parts[0]
        test_mark = int(parts[1])
        assignment_mark = int(parts[2])
        lab_mark = int(parts[3])

        # Store marks and total sum (total sum is directly proportional to average)
        total_sum = test_mark + assignment_mark + lab_mark
        students[name] = {
            'total': total_sum,
            'assignment': assignment_mark,
            'lab': lab_mark
        }

    if not students:
        return

    names = list(students.keys())

    # 1. Identify the student(s) with the highest average score (highest total sum)
    max_total = max(s['total'] for s in students.values())
    highest_avg_students = sorted([name for name in names if students[name]['total'] == max_total])
    print(" ".join(highest_avg_students))

    # 2. Identify the student(s) with the highest Assignment marks
    max_assign = max(s['assignment'] for s in students.values())
    highest_assign_students = sorted([name for name in names if students[name]['assignment'] == max_assign])
    print(" ".join(highest_assign_students))

    # 3. Identify the student(s) with the lowest lab marks
    min_lab = min(s['lab'] for s in students.values())
    lowest_lab_students = sorted([name for name in names if students[name]['lab'] == min_lab])
```

	Input	Expected	Got	
✓	4 James 67 89 56 Lalith 89 45 45 Ram 89 89 89 Sita 70 70 70	Ram James Ram Lalith Lalith	Ram James Ram Lalith Lalith	✓
✓	3 Raja 95 67 90 Aarav 89 90 90 ShadhanA 95 95 91	ShadhanA ShadhanA Aarav Raja Raja	ShadhanA ShadhanA Aarav Raja Raja	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Give a dictionary with value lists, sort the keys by summation of values in value list.

Input : test_dict = {'Gfg' : [6, 7, 4], 'best' : [7, 6, 5]}

Output : {'Gfg': 17, 'best': 18}

Explanation : Sorted by sum, and replaced.

Input : test_dict = {'Gfg' : [8,8], 'best' : [5,5]}

Output : {'best': 10, 'Gfg': 16}

Explanation : Sorted by sum, and replaced.

Sample Input:

2

Gfg 6 7 4

Best 7 6 5

Sample Output

Gfg 17

Best 18

For example:

Input	Result
2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Clean and split all lines
    lines = [line.strip() for line in input_data.splitlines() if line.strip()]
    if not lines:
        return

    n = int(lines[0])
    test_dict = {}

    # Read the dictionary items from input
    for i in range(1, n + 1):
        if i >= len(lines):
            break
        parts = lines[i].split()
        if not parts:
            continue
        key = parts[0]
        values = [int(x) for x in parts[1:]]
        test_dict[key] = values

    # Sort the dictionary items by the sum of their value lists
    sorted_dict = {
        key: sum(val)
        for key, val in sorted(test_dict.items(), key=lambda item: sum(item[1]))
    }

    # Print each key and its corresponding sum
    for key, total in sorted_dict.items():
        print(f"{key} {total}")
```

```
36
37 if __name__ == '__main__':
38     main()
```

	Input	Expected	Got	
✓	2 Gfg 6 7 4 Best 7 6 5	Gfg 17 Best 18	Gfg 17 Best 18	✓
✓	2 Gfg 6 6 Best 5 5	Best 10 Gfg 12	Best 10 Gfg 12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

In the game of Scrabble™, each letter has points associated with it. The total score of a word is the sum of the scores of its letters. More common letters are worth fewer points while less common letters are worth more points. The points associated with each letter are shown below:

Points Letters

1 A, E, I, L, N, O, R, S, T and U

2 D and G

3 B, C, M and P

4 F, H, V, W and Y

5 K

8 J and X

10 Q and Z

Write a program that computes and displays the Scrabble™ score for a word. Create a dictionary that maps from letters to point values. Then use the dictionary to compute the score.

A Scrabble™ board includes some squares that multiply the value of a letter or the value of an entire word. We will ignore these squares in this exercise.

Sample Input

REC

Sample Output

REC is worth 5 points.

For example:

Input	Result
REC	REC is worth 5 points.

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input word from standard input
    word = sys.stdin.read().strip()
    if not word:
        return

    # Dictionary mapping each letter to its Scrabble point value
    scrabble_scores = {
        'A': 1, 'E': 1, 'I': 1, 'L': 1, 'N': 1, 'O': 1, 'R': 1, 'S': 1, 'T': 1, 'U': 1,
        'D': 2, 'G': 2,
        'B': 3, 'C': 3, 'M': 3, 'P': 3,
        'F': 4, 'H': 4, 'V': 4, 'W': 4, 'Y': 4,
        'K': 5,
        'J': 8, 'X': 8,
        'Q': 10, 'Z': 10
    }

    # Calculate the total score by looking up each letter (case-insensitive)
    total_score = sum(scrabble_scores.get(char.upper(), 0) for char in word)

    # Print the result in the specified format
    print(f"{word} is worth {total_score} points.")

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	GOD	GOD is worth 5 points.	GOD is worth 5 points.	✓
✓	REC	REC is worth 5 points.	REC is worth 5 points.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a number, convert it into corresponding alphabet.

Input	Output
1	A
26	Z
27	AA
676	YZ

Input Format

Input is an integer

Output Format

Print the alphabets

Constraints

1 <= num <= 4294967295

Sample Input 1

26

Sample Output 1

Z

For example:

Test	Result
`print(excelNumber(26))`	Z

Answer: (penalty regime: 0 %)

Reset answer

```
def excelNumber(n: int) -> str:
    result = []
    while n > 0:
        n -= 1  # Adjust for 1-indexed conversion (A-Z)
        remainder = n % 26
        result.append(chr(ord('A') + remainder))
        n //= 26
    return "".join(reversed(result))

# If tested via standard input, you can also handle it like this:
if __name__ == '__main__':
    import sys
    input_data = sys.stdin.read().strip()
    if input_data:
        try:
            num = int(input_data)
            print(excelNumber(num))
        except ValueError:
            pass
```

	Test	Expected	Got	
✓	`print(excelNumber(26))`	Z	Z	✓

	Test	Expected	Got	
✓	`print(excelNumber(27))`	AA	AA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an array of names of candidates in an election. A candidate name in the array represents a vote cast to the candidate. Print the name of candidates received Max vote. If there is tie, print a lexicographically smaller name.

Examples:

Input : votes[] = {"john", "johnny", "jackie",
"johnny", "john", "jackie",
"jamie", "jamie", "john",
"johnny", "jamie", "johnny",
"john"};

Output : John

We have four Candidates with name as 'John', 'Johnny', 'jamie', 'jackie'. The candidates John and Johny get maximum votes. Since John is alphabetically smaller, we print it. Use dictionary to solve the above problem

Sample Input:

10
John
John
Johny
Jamie
Jamie
Johny
Jack
Johny
Johny
Jackie

Sample Output:

Johny

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    lines = [line.strip() for line in input_data.splitlines() if line.strip()]
    if not lines:
        return
```

```
    n = int(lines[0])
    votes = lines[1:n+1]

    # Use a dictionary to count votes for each candidate
    vote_count = {}
    for candidate in votes:
        vote_count[candidate] = vote_count.get(candidate, 0) + 1

    max_votes = max(vote_count.values())

    # Collect all candidates who achieved the maximum votes
    top_candidates = [candidate for candidate, count in vote_count.items() if count == max_votes]

    # If there is a tie, pick the lexicographically smaller name
    winner = min(top_candidates)

    print(winner)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	10 John John Johny Jamie Jamie Johny Jack Johny Johny Jackie	Johny	Johny	✓
✓	6 Ida Ida Ida Kiruba Kiruba Kiruba	Ida	Ida	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

You are given a string `word`. A letter is called **special** if it appears both in lowercase and uppercase in `word`.

Your task is to return the number of **special** letters in `word`.

Constraints

- The input string `word` will contain only alphabetic characters (both lowercase and uppercase).
- The solution must utilize a dictionary to determine the number of special letters.
- The function should handle various edge cases, such as strings without any special letters, strings with only lowercase or uppercase letters, and mixed strings.

Examples

Example 1:

Input: word = "aaAbcBC"
Output: 3

Explanation:
The special characters in `word` are 'a', 'b', and 'c'.

Example 2:

Input: word = "abc"
Output: 0

Explanation:
No character in `word` appears in uppercase.

For example:

Test	Result
`print(count_special_letters("AaBbCcDdEe"))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
def count_special_letters(word: str) -> int:
    # Dictionary mapping each lowercase letter to [has_lowercase, has_uppercase]
    char_dict = {}

    for ch in word:
        if ch.islower():
            if ch not in char_dict:
                char_dict[ch] = [False, False]
            char_dict[ch][0] = True  # Mark lowercase as present
        elif ch.isupper():
            lower_ch = ch.lower()
            if lower_ch not in char_dict:
                char_dict[lower_ch] = [False, False]
            char_dict[lower_ch][1] = True  # Mark uppercase as present

    # Count letters that have both lowercase and uppercase forms
    special_count = 0
    for status in char_dict.values():
        if status[0] and status[1]:
            special_count += 1

    return special_count

if __name__ == '__main__':
    import sys
    input_data = sys.stdin.read().strip()
    if input_data:
        print(count_special_letters(input_data))
```

	Test	Expected	Got	
✓	print(count_special_letters("AaBbCcDdEe"))	5	5	✓
✓	print(count_special_letters("ABCDE"))	0	0	✓
✓	print(count_special_letters("abcde"))	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

A company wants to send its quotation secretly to its client. The company decided to encrypt the amount they are sending to their client with some special symbols so that the equation amount will not be revealed to any external person. They used the special symbols !,@,#,$,%,^,&,*,>,< for 0,1,2,3,4,5,6,7,8,9 respectively. Write a python code to help the company to convert the amount to special symbols.

(Value rounded off to 2 decimal points)

Input

n: Float data type which reads amount to send

Output

s: : String data type which displays symbols

Sample Testcase 1

Input

10000

Output

@!!!!.!!

Sample Testcase2

1234.56

Output

@#$%.^&

For example:

Input	Result
1345.23	@$%^.#$
15000.59	@^!!!.^<
156789	@^&*><.!!

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input amount
    input_data = sys.stdin.read().strip()
    if not input_data:
        return

    # Convert to float and format to 2 decimal places
    n = float(input_data)
    formatted_str = f"{n:.2f}"

    # Define the symbol mapping for digits 0-9 and the decimal point
    symbol_map = {
        '0': '!',
        '1': '@',
        '2': '#',
        '3': '$',
        '4': '%',
        '5': '^',
        '6': '&',
        '7': '*',
        '8': '>',
        '9': '<',
        '.': '.'
    }

    # Translate each character in the formatted string using the mapping
    encrypted_amount = "".join(symbol_map.get(char, char) for char in formatted_str)

    # Print the resulting encrypted string
    print(encrypted_amount)

if __name__ == '__main__':
```

```
35 |    main()
```

	Input	Expected	Got	
✓	1345.23	@$%^.#$	@$%^.#$	✓
✓	15000.59	@^!!!.^<	@^!!!.^<	✓
✓	1234	@#$%.!!	@#$%.!!	✓
✓	156789	@^&*><.!!	@^&*><.!!	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.